

Deep Residual Neural Network (ResNet)-Based Adaptive Control: A Lyapunov-Based Approach

Omkar Sudhir Patil, Duc M. Le, Emily J. Griffis, and Warren E. Dixon

Abstract—Deep Neural Network (DNN)-based controllers have emerged as a tool to compensate for unstructured uncertainties in nonlinear dynamical systems. A recent breakthrough in the adaptive control literature provides a Lyapunov-based approach to derive weight adaptation laws for each layer of a fully-connected feedforward DNN-based adaptive controller. However, deriving weight adaptation laws from a Lyapunov-based analysis remains an open problem for deep residual neural networks (ResNets). This paper provides the first result on Lyapunov-derived adaptation laws for the weights of each layer of a ResNet-based adaptive controller. A nonsmooth Lyapunov-based analysis is provided to guarantee global asymptotic tracking error convergence.

I. INTRODUCTION

Deep Neural Network (DNN)-based controllers have emerged as a tool to compensate for unstructured uncertainties in nonlinear dynamical systems. The success of DNN-based controllers is powered by the ability of a DNN to approximate any continuous function over a compact domain [1]. A popular DNN-based control method is to first perform a DNN-based offline system identification using sampled input-output datasets that are collected by conducting experiments [2, Sec. 6.6]. Then, using the identified DNN, a feedforward term is constructed to compensate for the uncertainty in the system. However, the DNN weight estimates are not updated in real-time, and hence, such approaches involve open-loop implementation of the DNN-based feedforward term. Consequently, the open-loop implementation lacks the stability and robustness guarantees that closed-loop approaches can provide.

Unlike offline methods, a closed-loop adaptive feedforward term can be developed by deriving real-time DNN weight adaptation laws from a Lyapunov-based stability analysis. Various classical results [3]–[7] use Lyapunov-based techniques to develop weight adaptation laws, but only for single-hidden-layer networks. These results do not provide weight adaptation laws for DNNs with more than one hidden layer, since there

are mathematical challenges posed by the nested and nonlinear parameterization of a DNN that preclude the development of inner-layer weight adaptation laws. Recent results such as [8]–[10] develop Lyapunov-based adaptation laws for the output-layer weights of a DNN. However, to update the inner-layer weights, results in [8]–[10] collect datasets over discrete time-periods and iteratively identify the inner-layer weights using offline training algorithms. Consequently, these results suffer from the same drawbacks as offline methods. To circumvent offline identification of the inner-layer weights, the result in [11] provides a real-time inner-layer weight adaptation scheme based on a modular approach. However, modular designs only offer constraints on the adaptation laws and do not provide constructive insights on designing the adaptation laws.

The recent work [12] provides the first result on Lyapunov-derived weight adaptation laws for each layer of a DNN-based adaptive controller. To address the challenges posed by the nested and nonlinearly-parameterized structure of the DNN, a recursive representation of the DNN is developed. Then, a first-order Taylor series approximation is recursively applied for each layer. Using a Lyapunov-based stability analysis, the inner- and outer-layer weight adaptation laws are designed to cancel the cross-terms that appear from the first-order Taylor series approximation. Although the result in [12] provides Lyapunov-derived weight adaptation laws for the DNN, the development is restricted to fully-connected DNNs.

There are several limitations associated with standard DNN architectures such as fully-connected and convolutional DNNs. Deeper networks typically suffer from the problem of vanishing or exploding gradients, i.e., the rate of learning using a gradient-based update rule is highly sensitive to the magnitude of DNN weights. The challenges faced from the vanishing or exploding gradient problem are ubiquitous to both offline training [13] and real-time weight adaptation [12]. Additionally, in applications such as image recognition, the performance of a DNN is found to initially improve upon increasing the depth of the DNN. However, as the depth exceeds a threshold, the performance rapidly degrades [14].

To overcome the vanishing or exploding gradient problem and the degradation of performance with the increasing depth of a DNN, results in [14] introduce shortcut connections across layers, i.e., a feedforward connection between layers that are separated by more than one layer. DNNs with a shortcut connection are known as deep residual neural net-

The authors are with the Department of Mechanical and Aerospace Engineering, University of Florida, Gainesville FL 32611-6250 USA. Email: {patilomkarsudhir, ledan50, emilygriffis00, wdixon}@ufl.edu.

This research is supported in part by Office of Naval Research Grant N00014-21-1-2481, AFOSR award number FA8651-21-F-1027, and FA9550-19-1-0169. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the sponsoring agency.

works (ResNets). Although there has been significant research across various applications involving ResNets [14]–[18], the approximation power of ResNets has not yet been explored for adaptive control problems. Developing a ResNet-based adaptive feedforward control term with real-time weight adaptation laws is an open problem. Although real-time weight adaptation laws are developed for fully-connected feedforward DNNs in [12], the shortcut connections in ResNets pose additional mathematical challenges.

This paper provides the first result on Lyapunov-derived adaptation laws for the weights of each layer of a ResNet-based adaptive controller for uncertain nonlinear systems. To overcome the mathematical challenges posed by the shortcut connection, the ResNet is expressed as a composition of blocks of fully-connected DNNs. Then, a constructive Lyapunov-based approach is provided to derive weight adaptation laws for the ResNet using the gradient of each fully-connected DNN block. A nonsmooth Lyapunov-based analysis is provided to guarantee global asymptotic tracking error convergence.

Notation and Preliminaries: The space of essentially bounded Lebesgue measurable functions is denoted by $\mathcal{L}_\infty$. The right-to-left matrix product operator is represented by $\prod$, i.e., $\prod_{p=1}^m A_p = A_m \dots A_2 A_1$ and $\prod_{p=a}^m A_p = 1$ if $a > m$. The Kronecker product is denoted by $\otimes$. Function compositions are denoted using the symbol $\circ$, e.g., $(g \circ h)(x) = g(h(x))$, given suitable functions g and h . The Filippov set-valued map defined in [19, Equation 2b] is denoted by $K[\cdot]$. The notation $\stackrel{a.a.t.}{\cdot}$ denotes that the relation $(\cdot)$ holds for almost all time (a.a.t.). Consider a Lebesgue measurable and locally essentially bounded function $h : \mathbb{R}^n \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$. Then, the function $y : \mathcal{I} \rightarrow \mathbb{R}^n$ is called a Filippov solution of $\dot{y} = h(y, t)$ on the interval $\mathcal{I} \subseteq \mathbb{R}_{\geq 0}$ if y is absolutely continuous on $\mathcal{I}$ and $\dot{y} \stackrel{a.a.t.}{\in} K[h](y, t)$. Given some functions f and g , the notation $f(y) = \mathcal{O}^m(g(y))$ means that there exists some constants $M \in \mathbb{R}_{>0}$ and $y_0 \in \mathbb{R}$ such that $\|f(y)\| \leq M \|g(y)\|^m$ for all $y \geq y_0$. Given some matrix $A \triangleq [a_{i,j}] \in \mathbb{R}^{n \times m}$, where $a_{i,j}$ denotes the element in the i^{th} row and j^{th} column of A , the vectorization operator is defined as $\text{vec}(A) \triangleq [a_{1,1}, \dots, a_{1,m}, \dots, a_{n,1}, \dots, a_{n,m}]^T \in \mathbb{R}^{nm}$. The p -norm is denoted by $\|\cdot\|_p$, where the subscript is suppressed when $p = 2$. The Frobenius norm is denoted by $\|\cdot\|_F \triangleq \|\text{vec}(\cdot)\|$. Given any $A \in \mathbb{R}^{p \times a}$, $B \in \mathbb{R}^{a \times r}$, and $C \in \mathbb{R}^{r \times s}$, the vectorization operator satisfies the property [20, Proposition 7.1.9]

$$\text{vec}(ABC) = (C^T \otimes A) \text{vec}(B). \quad (1)$$

Differentiating (1) on both sides with respect to $\text{vec}(B)$ yields the property

$$\frac{\partial}{\partial \text{vec}(B)} \text{vec}(ABC) = (C^T \otimes A). \quad (2)$$

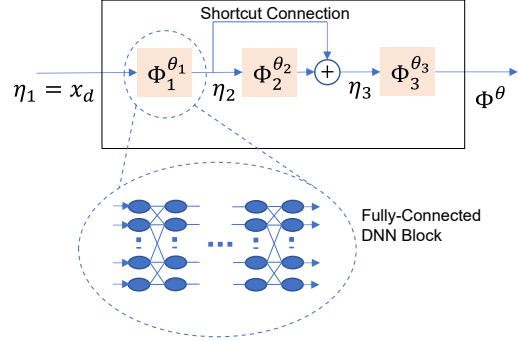


Figure 1. Illustration of the ResNet architecture in (6). The ResNet is modeled as a composition of 3 fully-connected DNN blocks denoted by $\Phi_p^{\theta_p}$ for all $p \in \{1, 2, 3\}$, where the input and the vector of weights of Φ_p are denoted by η_p and θ_p , respectively. The ResNet involves a shortcut connection from $\Phi_1^{\theta_1}$ to $\Phi_3^{\theta_3}$ that can be represented using $\eta_3 = \eta_2 + \Phi_2^{\theta_2}(\eta_2)$.

II. UNKNOWN SYSTEM DYNAMICS AND CONTROL DESIGN

Consider a control-affine nonlinear dynamic system modeled as

$$\dot{x} = f(x) + u, \quad (3)$$

where $x : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$ denotes a solution to (3), $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ denotes an unknown differentiable drift vector field, and $u : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$ denotes a control input. Let the tracking error $e : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$ be defined as

$$e \triangleq x - x_d, \quad (4)$$

where $x_d : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$ denotes a continuously differentiable reference trajectory. The reference trajectory is designed such that $x_d(t) \in \Omega \forall t \in \mathbb{R}_{\geq 0}$ and $\dot{x}_d \in \mathcal{L}_\infty$, where $\Omega \subset \mathbb{R}^n$ denotes a known compact set. The control objective is to design a ResNet-based adaptive controller that achieves global asymptotic tracking error convergence.

A. ResNet Architecture

The unknown drift vector field f can be approximated using a ResNet. A ResNet can be modeled using blocks of fully-connected DNNs as shown in Figure 1 [14]. For the simplicity of notation in the subsequent development, this paper considers ResNets that can be represented using three fully-connected DNN blocks. The subsequent development can be extended for other ResNet configurations on a case-by-case basis.

Let $\Phi_p : \mathbb{R}^{L_{p,0}} \times \mathbb{R}^{L_{p,0} \times L_{p,1}} \times \dots \times \mathbb{R}^{L_{p,k_p} \times L_{p,k_p+1}} \rightarrow \mathbb{R}^{L_{p,k_p+1}}$ denote the p^{th} fully-connected DNN block which is defined as $\Phi_p(\eta_p, V_{p,0}, \dots, V_{p,k_p}) \triangleq (V_{p,k_p}^T \phi_{p,k_p} \circ \dots \circ V_{p,1}^T \phi_{p,1})(V_{p,0}^T \eta_p)$ for all $p \in \{1, 2, 3\}$, with input $\eta_p \in \mathbb{R}^{L_{p,0}}$, where $k_p \in \mathbb{Z}_{>0}$ denotes the number of hidden layers in Φ_p . Additionally, let $L_{p,j} \in \mathbb{N}$ and $V_{p,j} \in \mathbb{R}^{L_{p,j} \times L_{p,j+1}}$ denote the number

of nodes and weight matrix in the j^{th} layer of Φ_p for all $(p, j) \in \{1, 2, 3\} \times \{0, \dots, k_p\}$, respectively. Let $\phi_{p,j} : \mathbb{R}^{L_{p,j}} \rightarrow \mathbb{R}^{L_{p,j}}$ denote the vector of smooth activation functions in the j^{th} layer of Φ_p for all $(p, j) \in \{1, 2, 3\} \times \{1, \dots, k_p\}$.¹ If the ResNet involves multiple types of activation functions at each layer, then $\phi_{p,j}$ may be represented as $\phi_{p,j} \triangleq [\varsigma_{p,j,1} \dots \varsigma_{p,j,L_j}]^T$, where $\varsigma_{p,j,i} : \mathbb{R} \rightarrow \mathbb{R}$ denotes the activation function at the i^{th} node of the j^{th} layer of Φ_p .² All the weights of Φ_p can be represented by the vector $\theta_p \triangleq [\text{vec}(V_{p,0})^T \text{vec}(V_{p,1})^T \dots \text{vec}(V_{p,k_p})^T]^T \in \mathbb{R}^{\sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}}$. The fully-connected block Φ_p can be expressed as $\Phi_p = V_{p,k_p}^T \varphi_{p,k_p}$, where $\varphi_{p,0} : \mathbb{R}^{L_{p,0}} \rightarrow \mathbb{R}^{L_{p,0}}$ and $\varphi_{p,j} : \mathbb{R}^{L_{p,0}} \times \mathbb{R}^{L_{p,0} \times L_{p,1}} \times \dots \times \mathbb{R}^{L_{p,j-1} \times L_{p,j}} \rightarrow \mathbb{R}^{L_{p,j}}$ $\forall j \in \{1, \dots, k_p\}$ denote the recursive relation defined as

$$\varphi_{p,j} \triangleq \begin{cases} \phi_{p,j} (V_{p,j-1}^T \varphi_{p,j-1}), & j \in \{1, \dots, k_p\}, \\ \eta_p, & j = 0. \end{cases} \quad (5)$$

The arguments of $\varphi_{p,j}$ are suppressed for notational brevity. Let $\phi'_{p,j} : \mathbb{R}^{L_{p,j}} \rightarrow \mathbb{R}^{L_{p,j} \times L_{p,j}}$ denote the Jacobian $\phi'_{p,j}(y) \triangleq \frac{\partial}{\partial y} \phi_{p,j}(y) \forall y \in \mathbb{R}^{L_{p,j}}$. The short-hand notations $\Phi_p^{\theta}(\eta_p) \triangleq \Phi_p(\eta_p, V_{p,0}, V_{p,1}, \dots, V_{p,k_p})$ and $\varphi'_{p,j} \triangleq \phi'_{p,j} (V_{p,j-1}^T \varphi_{p,j-1})$ are defined for notational brevity in the subsequent development. Additionally, let $\theta \triangleq [\theta_1^T \theta_2^T \theta_3^T]^T \in \mathbb{R}^{\sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}}$ denote the vector of weights for the entire ResNet. The ResNet $(\eta_1, \theta) \mapsto \Phi^{\theta}(\eta_1)$ is modeled as [14]

$$\Phi^{\theta}(\eta_1) \triangleq \Phi_3^{\theta_3} \left(\Phi_2^{\theta_2} \left(\Phi_1^{\theta_1}(\eta_1) \right) + \Phi_1^{\theta_1}(\eta_1) \right), \quad (6)$$

where $\Phi : \mathbb{R}^n \times \mathbb{R}^{\sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}} \rightarrow \mathbb{R}^n$ (see Figure 1). For the ResNet in (6), $\eta_1 \triangleq x_d$, $\eta_2 \triangleq \Phi_1^{\theta_1}(\eta_1)$, and $\eta_3 \triangleq \Phi_1^{\theta_1}(\eta_1) + \Phi_2^{\theta_2}(\eta_2)$. Then, it follows that $\Phi^{\theta}(x_d) \triangleq \Phi_3^{\theta_3}(\eta_3)$. The fully-connected DNN blocks must satisfy the constraints $L_{1,0} = n$, $L_{3,k_3+1} = n$, and $L_{1,k_1+1} = L_{2,0} = L_{2,k_2+1} = L_{3,0}$.

Remark 1. A fully-connected DNN is a special case of the ResNet in (6). This can be established by selecting Φ_2 to be identically zero over its domain, which yields $\Phi^{\theta}(\eta_1) = \Phi_3^{\theta_3}(\Phi_1^{\theta_1}(\eta_1))$.

To facilitate the subsequent development, the following assumption is made.

Assumption 1. The function space of ResNets given by (6) is dense in $\mathcal{C}(\Omega)$, where $\mathcal{C}(\Omega)$ denotes the space of functions continuous over Ω .

¹For the case of DNNs with nonsmooth activation functions, the reader is referred to [12] where a switched analysis is provided to account for the nonsmooth nature of activation functions. To better focus on our main contribution without loss of generality, we restrict our attention to smooth activation functions.

²The bias terms are omitted for simplicity of the notation.

Remark 2. Assumption 1 implies that ResNets satisfy the universal function approximation property that is well-known for various DNN architectures [1]. The universal function approximation property of ResNets is a common assumption that is widely used in the deep learning literature, and has been rigorously established for ResNets with specific activation functions in [21] and [22].

Consider any vector field $f \in \mathcal{C}(\Omega)$ and a prescribed accuracy $\bar{\varepsilon} \in \mathbb{R}_{>0}$. Then, by Assumption 1, for all $(p, j) \in \{1, 2, 3\} \times \{0, \dots, k_p\}$, there exist $k_p, L_{p,j} \in \mathbb{Z}_{>0}$ and a corresponding vector of ideal weights $\theta^* \in \mathbb{R}^{\sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}}$ such that $\sup_{x_d \in \Omega} \|f(x_d) - \Phi^{\theta^*}(x_d)\| \leq \bar{\varepsilon}$. Therefore, the drift vector field $x_d \mapsto f(x_d)$ can be modeled as

$$f(x_d) = \Phi^{\theta^*}(x_d) + \varepsilon(x_d), \quad (7)$$

where θ^* denotes the vector of ideal weights, and $\varepsilon : \mathbb{R}^n \rightarrow \mathbb{R}^n$ denotes an unknown function reconstruction error that can be bounded as $\sup_{x_d \in \Omega} \|\varepsilon(x_d)\| \leq \bar{\varepsilon}$. To facilitate the subsequent analysis, the following assumption is made (cf., [23, Assumption 1]).

Assumption 2. There exists a known constant $\bar{\theta} \in \mathbb{R}_{>0}$ such that $\|\theta^*\| \leq \bar{\theta}$.

B. Adaptation Laws

The ResNet-based model in (7) can be leveraged to approximate the unknown drift vector field f . However, since the ideal weights are unknown, adaptive estimates are developed. The adaptive weight estimate for the j^{th} layer of Φ_p is denoted by $\hat{V}_{p,j} : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^{L_{p,j} \times L_{p,j+1}} \forall (p, j) \in \{1, 2, 3\} \times \{0, \dots, k_p\}$. Then, $\hat{\theta}_p : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^{\sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}}$ is defined as $\hat{\theta}_p = [\text{vec}(\hat{V}_{p,0})^T, \dots, \text{vec}(\hat{V}_{p,k_p})^T]^T$ for all $p \in \{1, 2, 3\}$, $\hat{\theta} : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^{\sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}}$ is defined as $\hat{\theta} \triangleq [\hat{\theta}_1^T \hat{\theta}_2^T \hat{\theta}_3^T]^T$, and the ResNet-based adaptive estimate of $f(x_d) \forall x_d \in \Omega$ is denoted by $\Phi^{\hat{\theta}}(x_d)$. The corresponding weight estimation error $\tilde{\theta} : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^{\sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}}$ is defined as $\tilde{\theta} \triangleq \theta^* - \hat{\theta}$. Based on the subsequent stability analysis, the adaptation law for the weight estimates of the ResNet in (6) is designed as

$$\dot{\hat{\theta}} \triangleq \text{proj}(\Gamma \Phi'^T e), \quad (8)$$

where $\Gamma \in \mathbb{R}^{\sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1} \times \sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}}$ denotes a positive-definite adaptation gain matrix, and $\Phi' \in \mathbb{R}^{n \times \sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}}$ is a short-hand notation denoting the Jacobian $\Phi' \triangleq \frac{\partial \Phi^{\hat{\theta}}(x_d)}{\partial \hat{\theta}}$. In (8), $\text{proj}(\cdot)$ denotes the projection operator defined in [24, Appendix E, Eq. E.4], which is used to ensure $\hat{\theta}(t) \in \mathcal{B} \triangleq \{\theta \in \mathbb{R}^{\sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}} : \|\theta\| \leq \bar{\theta}\} \forall t \in \mathbb{R}_{\geq 0}$. The Jacobian term Φ' can be evaluated as follows. Let $\hat{\eta}_1 \in \mathbb{R}^{L_{1,0}}$, $\hat{\eta}_2 \in \mathbb{R}^{L_{2,0}}$, and $\hat{\eta}_3 \in \mathbb{R}^{L_{3,0}}$ be defined as $\hat{\eta}_1 \triangleq x_d$, $\hat{\eta}_2 \triangleq \Phi_1^{\hat{\theta}_1}(\hat{\eta}_1)$, and $\hat{\eta}_3 \triangleq \Phi_1^{\hat{\theta}_1}(\hat{\eta}_1) + \Phi_2^{\hat{\theta}_2}(\hat{\eta}_2)$, respectively.

Then, it follows that $\Phi^{\hat{\theta}}(x_d) = \Phi_3^{\hat{\theta}_3}(\hat{\eta}_3)$. To facilitate the subsequent analysis, the short-hand notations $\Lambda_p \triangleq \frac{\partial \Phi_p^{\hat{\theta}_p}(\hat{\eta}_p)}{\partial \hat{\theta}_p}$, $\Lambda_{p,j} \triangleq \frac{\partial \Phi_p^{\hat{\theta}_p}(\hat{\eta}_p)}{\partial \text{vec}(\hat{V}_{p,j})}$, and $\Phi'_p \triangleq \frac{\partial \Phi_p^{\hat{\theta}_p}(\hat{\eta}_p)}{\partial \hat{\eta}_p}$ are introduced. Since $\Phi' = \left[\left(\frac{\partial \Phi^{\hat{\theta}}(x_d)}{\partial \hat{\theta}_1} \right), \left(\frac{\partial \Phi^{\hat{\theta}}(x_d)}{\partial \hat{\theta}_2} \right), \left(\frac{\partial \Phi^{\hat{\theta}}(x_d)}{\partial \hat{\theta}_3} \right) \right]$, it can be verified using the chain rule that

$$\Phi' \triangleq \begin{bmatrix} \Phi'_3 (\Phi'_2 + I_{L_{3,0}}) \Lambda_1 & \Phi'_3 \Lambda_2 & \Lambda_3 \end{bmatrix}. \quad (9)$$

In (9), the Jacobian terms Λ_p and Φ'_p can be computed for all $p \in \{1, 2, 3\}$ as follows. Since $\hat{\theta}_p = [\text{vec}(\hat{V}_{p,0})^T, \dots, \text{vec}(\hat{V}_{p,k_p})^T]^T$, it follows that $\frac{\partial \Phi_p^{\hat{\theta}_p}(\hat{\eta}_p)}{\partial \hat{\theta}_p} \triangleq \left[\left(\frac{\partial \Phi_p^{\hat{\theta}_p}(\hat{\eta}_p)}{\partial \text{vec}(\hat{V}_{p,0})} \right), \dots, \left(\frac{\partial \Phi_p^{\hat{\theta}_p}(\hat{\eta}_p)}{\partial \text{vec}(\hat{V}_{p,k_p})} \right) \right]$. Therefore, using the definitions of Λ_p and $\Lambda_{p,j}$ yields

$$\Lambda_p = [\Lambda_{p,0} \quad \Lambda_{p,1} \quad \dots \quad \Lambda_{p,k_p}] , \forall p \in \{1, 2, 3\}. \quad (10)$$

For brevity in the subsequent analysis, the short-hand notations $\hat{\varphi}_{p,j} \triangleq \varphi_{p,j}(\hat{\eta}_p, \hat{V}_{p,0}, \dots, \hat{V}_{p,j})$ and $\hat{\varphi}'_{p,j} \triangleq \varphi'_{p,j}(\hat{\eta}_p, \hat{V}_{p,0}, \dots, \hat{V}_{p,j})$ are introduced. Using (5), the chain rule, and the property of vectorization operators in (2), the terms $\Lambda_{p,0}$ and $\Lambda_{p,j}$ can be expressed as

$$\Lambda_{p,0} = \left(\prod_{l=1}^{\hat{k}_p} \hat{V}_{p,l}^T \hat{\varphi}'_{p,l} \right) (I_{L_{p,1}} \otimes \hat{\eta}_p^T), \quad (11)$$

and

$$\Lambda_{p,j} = \left(\prod_{l=j+1}^{\hat{k}_p} \hat{V}_{p,l}^T \hat{\varphi}'_{p,l} \right) (I_{L_{p,j+1}} \otimes \hat{\varphi}_{p,j}^T), \quad (12)$$

for all $j \in \{1, \dots, k_p\}$, respectively. Similarly, the term Φ'_p can be expressed as

$$\Phi'_p = \left(\prod_{l=1}^{\hat{k}_p} \hat{V}_{p,l}^T \hat{\varphi}'_{p,l} \right) \hat{V}_{p,0}^T. \quad (13)$$

Remark 3. As discussed in Remark 1, fully-connected DNNs are a special case of ResNets. Hence, it can be verified that the weight adaptation laws developed in (8)-(13) are a special case of the adaptation laws developed in [12] for fully-connected DNNs.

Remark 4. If Φ_2 suffers from the vanishing gradient problem, i.e., $\|\Phi_2\|_F \approx 0$, then $\Phi'_3 (\Phi'_2 + I_{L_{3,0}}) \Lambda_1 \approx \Phi'_3 \Lambda_1$. Thus, the term $\Phi'_3 (\Phi'_2 + I_{L_{3,0}}) \Lambda_1$ in (9) indicates the shortcut connection circumvents the vanishing gradient problem in the ResNet when Φ_2 has a vanishing gradient. Additionally, vanishing gradients can be avoided in Φ_1 and Φ_3 by selecting smaller values of k_1 and k_3 . Therefore, given different ResNets with the same depth and width, ResNets with shortcut connections across more layers, i.e., ResNet configurations with lower k_1

and k_3 and higher k_2 can better mitigate the vanishing gradient problem.

C. Control Law Development

Let the function $f_e : \mathbb{R}^n \times \Omega \rightarrow \mathbb{R}^n$ be defined as $f_e \triangleq f(x) - f(x_d)$. Using [25, Lemma 5], the function $(x, x_d) \mapsto f_e$ can be bounded as $\|f_e\| \leq \rho(\|e\|)\|e\|$, for all $x \in \mathbb{R}^n$ and $x_d \in \Omega$, where $\rho : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ denotes a known strictly increasing function. Based on the subsequent stability analysis, the control input is designed as

$$u \triangleq \dot{x}_d - \Phi^{\hat{\theta}}(x_d) - \rho(\|e\|)e - \sigma_e e - \sigma_s \text{sgn}(e), \quad (14)$$

where $\sigma_e, \sigma_s \in \mathbb{R}_{>0}$ are constant control gains, and $\text{sgn}(\cdot)$ denotes the vector signum function.

Taking the time-derivative of (4) and substituting in (3) and (14) yields the closed-loop error system

$$\dot{e} = f(x) - \Phi^{\hat{\theta}}(x_d) - \rho(\|e\|)e - \sigma_e e - \sigma_s \text{sgn}(e). \quad (15)$$

Adding and subtracting $f(x_d)$ and substituting in (7), the closed-loop error system in (15) can be expressed as

$$\begin{aligned} \dot{e} &= f_e + \Phi^{\theta^*}(x_d) - \Phi^{\hat{\theta}}(x_d) + \varepsilon(x_d) \\ &\quad - \rho(\|e\|)e - \sigma_e e - \sigma_s \text{sgn}(e). \end{aligned} \quad (16)$$

The ResNet in (6) is nonlinearly parameterized in terms of the weights. Adaptive control design for nonlinearly parameterized systems is known to be a difficult problem [26]. A number of methods have been developed in the adaptive control literature to address the challenges posed by a nonlinear parameterization [3], [12], [26], [27]. In particular, first-order Taylor series approximation-based techniques have shown promising results for neural network-based adaptive controllers [3], [5], [12]. Specifically, the result in [12] leveraged a first-order Taylor series approximation to derive weight adaptation laws for a fully-connected DNN-based adaptive controller. Thus, motivation exists to explore a Taylor series approximation-based design to derive adaptation laws for the ResNet. For the ResNet given by (6), a first-order Taylor series approximation-based error model is given by [23, Eq. 22]

$$\Phi^{\theta^*}(x_d) - \Phi^{\hat{\theta}}(x_d) = \Phi' \tilde{\theta} + \mathcal{O}^2(\|\tilde{\theta}\|), \quad (17)$$

where $\mathcal{O}^2(\|\tilde{\theta}\|)$ denotes the higher-order terms. Since $\|\tilde{\theta}\| \leq \bar{\theta}$ by the use of projection operator, it follows from Assumption 2 that $\|\tilde{\theta}\| \leq \|\theta^*\| + \|\hat{\theta}\| \leq 2\bar{\theta}$. Due to the facts that the ResNet is smooth, $x_d \in \Omega$, and $\|\tilde{\theta}\| \leq 2\bar{\theta}$, it can be shown that there exists a known constant $\bar{\Delta} \in \mathbb{R}_{>0}$ such that $\|\mathcal{O}^2(\|\tilde{\theta}\|)\| \leq \bar{\Delta}$ (cf., [12, Eq. 18]). Then, substituting (17) into (16) yields

$$\begin{aligned} \dot{e} &= f_e + \Phi' \tilde{\theta} + \mathcal{O}^2(\|\tilde{\theta}\|) + \varepsilon(x_d) \\ &\quad - \rho(\|e\|)e - \sigma_e e - \sigma_s \text{sgn}(e). \end{aligned} \quad (18)$$

To facilitate the subsequent analysis, let $z \triangleq [e^T, \tilde{\theta}^T]^T \in \mathbb{R}^\Psi$ denote a concatenated state, where $\Psi \triangleq n + \sum_{p=1}^3 \sum_{j=0}^{k_p} L_{p,j} L_{p,j+1}$. Then, using (8) and (18) yields the dynamics

$$\dot{z} = h(z, t), \quad (19)$$

where $h : \mathbb{R}^\Psi \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^\Psi$ is defined as

$$h(z, t) \triangleq \begin{bmatrix} f_e + \Phi' \tilde{\theta} + \mathcal{O}^2(\|\tilde{\theta}\|) + \varepsilon(x_d) \\ -\rho(\|e\|)e - \sigma_e e - \sigma_s \text{sgn}(e) \\ -\text{proj}(\Gamma \Phi'^T e) \end{bmatrix}. \quad (20)$$

III. STABILITY ANALYSIS

Based on the nonsmooth analysis technique in [28], the following theorem establishes the invariance properties of Filippov solutions to (19) and provides guarantees of global asymptotic tracking error convergence for the system in (3).

Theorem 1. *For the dynamical system in (3), the controller in (14) and the adaptation law in (8) ensure global asymptotic tracking error convergence in the sense that $z \in \mathcal{L}_\infty$ and $\lim_{t \rightarrow \infty} \|e(t)\| = 0$, provided Assumptions 1 and 2 hold, and the following gain condition is satisfied:*

$$\sigma_s > \bar{\varepsilon} + \bar{\Delta}. \quad (21)$$

Proof: Consider the candidate Lyapunov function $\mathcal{V}_L : \mathbb{R}^\Psi \rightarrow \mathbb{R}_{\geq 0}$ defined as

$$\mathcal{V}_L(z) \triangleq \frac{1}{2} e^T e + \frac{1}{2} \tilde{\theta}^T \Gamma^{-1} \tilde{\theta}, \quad (22)$$

which satisfies the inequality $\underline{\alpha} \|z\|^2 \leq \mathcal{V}_L(z) \leq \bar{\alpha} \|z\|^2$, where $\underline{\alpha}, \bar{\alpha} \in \mathbb{R}_{\geq 0}$ are known constants. Let $\partial \mathcal{V}_L$ denote the Clarke gradient of $\mathcal{V}_L$ defined in [29, p. 39]. Since $z \mapsto \mathcal{V}_L(z)$ is continuously differentiable, $\partial \mathcal{V}_L(z) = \{\nabla \mathcal{V}_L(z)\}$, where ∇ denotes the standard gradient operator. Based on (20) and the chain rule in [30, Thm 2.2], it can be verified that $t \rightarrow \mathcal{V}_L(z(t))$ satisfies the differential inclusion

$$\begin{aligned} \dot{\mathcal{V}}_L &\stackrel{a.a.t.}{\in} \bigcap_{\xi \in \partial \mathcal{V}_L(z)} \xi^T K[h](z, t) \\ &= \nabla \mathcal{V}_L(z)^T K[h](z, t) \\ &= e^T \left(f_e + \mathcal{O}^2(\|\tilde{\theta}\|) + \varepsilon(x_d) - \rho(\|e\|)e \right) \\ &\quad - \sigma_e \|e\|^2 - \sigma_s e^T K[\text{sgn}](e) \\ &\quad + e^T \Phi' \tilde{\theta} - \tilde{\theta}^T \Gamma^{-1} K[\text{proj}](\Gamma \Phi'^T e). \end{aligned} \quad (23)$$

Using [24, Lemma E.1.IV] and the fact that $K[\text{proj}](\cdot)$ is the set of convex combinations of $\text{proj}(\cdot)$ and $(\cdot)$, the term with the projection operator in (23) can be bounded as

$$-\tilde{\theta}^T \Gamma^{-1} K[\text{proj}](\Gamma \Phi'^T e) \leq -\tilde{\theta}^T \Phi'^T e. \quad (24)$$

Using (24) and the facts that $e^T K[\text{sgn}](e) = \|e\|_1$ and $e^T f_e \leq \rho(\|e\|) \|e\|^2$, (23) can be bounded as

$$\dot{\mathcal{V}}_L \stackrel{a.a.t.}{\leq} -\sigma_e \|e\|^2 + e^T \left(\mathcal{O}^2(\|\tilde{\theta}\|) + \varepsilon(x_d) \right) - \sigma_s \|e\|_1. \quad (25)$$

Based on Holder's inequality, triangle inequality, and the fact that $\|e\| \leq \|e\|_1$, the following inequality can be obtained: $e^T \left(\mathcal{O}^2(\|\tilde{\theta}\|) + \varepsilon(x_d) \right) \leq \|e\|_1 \left(\|\mathcal{O}^2(\|\tilde{\theta}\|)\| + \|\varepsilon(x_d)\| \right) \leq (\bar{\varepsilon} + \bar{\Delta}) \|e\|_1$. Then, provided the gain condition in (21) is satisfied, the right-hand side of (25) can be upper-bounded as

$$\dot{\mathcal{V}}_L \stackrel{a.a.t.}{\leq} -\sigma_e \|e\|^2. \quad (26)$$

Based on (26), invoking [28, Corollary 1] yields $z \in \mathcal{L}_\infty$ and $\lim_{t \rightarrow \infty} \|e(t)\| = 0$. Additionally, due to the facts that $(x_d, \hat{\theta}) \rightarrow \Phi^{\hat{\theta}}(x_d)$ is smooth, $x_d \in \Omega$, and $\hat{\theta} \in \mathcal{B}$, it follows that $\Phi^{\hat{\theta}}(x_d)$ is bounded. Since each term on the right-hand side of (14) is bounded, the control input $u \in \mathcal{L}_\infty$. Since $\phi_{p,j}$ and $\phi'_{p,j}$ are smooth for all $(p, j) \in \{1, 2, 3\} \times \{0, \dots, k_p\}$, it follows from (9)-(13) that Φ' is bounded. Then, every term on the right-hand side of (8) is bounded. Hence, $\hat{\theta}$ is bounded. ■

IV. CONCLUSION

This paper provides the first result on Lyapunov-derived adaptation laws for the weights of each layer of a ResNet-based adaptive controller. Future work may involve the application of ResNets for online system identification and state estimation problems. Additionally, more complicated architectures of ResNets such as recurrent ResNets can be explored for control applications.

REFERENCES

- [1] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, pp. 359–366, 1985.
- [2] S. L. Brunton and J. N. Kutz, *Data-driven science and engineering: Machine learning, dynamical systems, and control*. Cambridge University Press, 2019.
- [3] F. Lewis, A. Yesildirek, and K. Liu, "Multilayer neural net robot controller: structure and stability proofs," *IEEE Trans. Neural Netw.*, vol. 7, no. 2, pp. 388–399, 1996.
- [4] F. L. Lewis, S. Jagannathan, and A. Yesildirak, *Neural network control of robot manipulators and nonlinear systems*. Philadelphia, PA: CRC Press, 1998.
- [5] S. S. Ge, C. C. Hang, T. H. Lee, and T. Zhang, *Stable Adaptive Neural Network Control*. Boston, MA: Kluwer Academic Publishers, 2002.
- [6] P. M. Patre, W. MacKunis, K. Kaiser, and W. E. Dixon, "Asymptotic tracking for uncertain dynamic systems via a multilayer neural network feedforward and RISE feedback control structure," *IEEE Trans. Autom. Control*, vol. 53, no. 9, pp. 2180–2185, 2008.
- [7] P. Patre, S. Bhasin, Z. D. Wilcox, and W. E. Dixon, "Composite adaptation for neural network-based controllers," *IEEE Trans. Autom. Control*, vol. 55, no. 4, pp. 944–950, 2010.
- [8] R. Sun, M. Greene, D. Le, Z. Bell, G. Chowdhary, and W. E. Dixon, "Lyapunov-based real-time and iterative adjustment of deep neural networks," *IEEE Control Syst. Lett.*, vol. 6, pp. 193–198, 2022.
- [9] G. Joshi and G. Chowdhary, "Deep model reference adaptive control," in *Proc. IEEE Conf. Decis. Control*, pp. 4601–4608, IEEE, 2019.
- [10] G. Joshi, J. Virdi, and G. Chowdhary, "Asynchronous deep model reference adaptive control," in *Conf. Robot Learn.*, 2020.
- [11] D. Le, M. Greene, W. Makumi, and W. E. Dixon, "Real-time modular deep neural network-based adaptive control of nonlinear systems," *IEEE Control Syst. Lett.*, vol. 6, pp. 476–481, 2022.

- [12] O. Patil, D. Le, M. Greene, and W. E. Dixon, "Lyapunov-derived control and adaptive update laws for inner and outer layer weights of a deep neural network," *IEEE Control Syst Lett.*, vol. 6, pp. 1855–1860, 2022.
- [13] I. Goodfellow, Y. Bengio, A. Courville, and Y. Bengio, *Deep Learning*, vol. 1. MIT press Cambridge, 2016.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 770–778, 2016.
- [15] Y. Tai, J. Yang, and X. Liu, "Image super-resolution via deep recursive residual network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 3147–3155, 2017.
- [16] J. Li, F. Fang, K. Mei, and G. Zhang, "Multi-scale residual network for image super-resolution," in *Proc. Eur. Conf. Comput. Vis.*, pp. 517–532, 2018.
- [17] M. Boroumand, M. Chen, and J. Fridrich, "Deep residual network for steganalysis of digital images," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 5, pp. 1181–1193, 2018.
- [18] T. Tan, Y. Qian, H. Hu, Y. Zhou, W. Ding, and K. Yu, "Adaptive very deep convolutional residual network for noise robust speech recognition," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 26, no. 8, pp. 1393–1405, 2018.
- [19] B. E. Paden and S. S. Sastry, "A calculus for computing Filippov's differential inclusion with application to the variable structure control of robot manipulators," *IEEE Trans. Circuits Syst.*, vol. 34, pp. 73–82, Jan. 1987.
- [20] D. S. Bernstein, *Matrix mathematics*. Princeton university press, 2009.
- [21] H. Lin and S. Jegelka, "ResNet with one-neuron hidden layers is a universal approximator," *Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [22] P. Tabuada and B. Gharesifard, "Universal approximation power of deep residual neural networks via nonlinear control theory," in *Int. Conf. Learn. Represent.*, 2020.
- [23] F. L. Lewis, A. Yegildirek, and K. Liu, "Multilayer neural-net robot controller with guaranteed tracking performance," *IEEE Trans. Neural Netw.*, vol. 7, pp. 388–399, Mar. 1996.
- [24] M. Krstic, I. Kanellakopoulos, and P. V. Kokotovic, *Nonlinear and Adaptive Control Design*. New York, NY, USA: John Wiley & Sons, 1995.
- [25] R. Kamalapurkar, J. A. Rosenfeld, J. Klotz, R. J. Downey, and W. E. Dixon, "Supporting lemmas for RISE-based control methods," arXiv:1306.3432, 2014.
- [26] A. M. Annaswamy, F. P. Skantze, and A.-P. Loh, "Adaptive control of continuous time systems with convex/concave parametrization," *Automatica*, vol. 34, no. 1, pp. 33–49, 1998.
- [27] S. B. Roy, S. Bhasin, and I. N. Kar, "Robust gradient-based adaptive control of nonlinearly parametrized plants," *IEEE Control Syst. Lett.*, vol. 1, no. 2, pp. 352–357, 2017.
- [28] N. Fischer, R. Kamalapurkar, and W. E. Dixon, "LaSalle-Yoshizawa corollaries for nonsmooth systems," *IEEE Trans. Autom. Control*, vol. 58, pp. 2333–2338, Sep. 2013.
- [29] F. H. Clarke, *Optimization and nonsmooth analysis*. SIAM, 1990.
- [30] D. Shevitz and B. Paden, "Lyapunov stability theory of nonsmooth systems," *IEEE Trans. Autom. Control*, vol. 39 no. 9, pp. 1910–1914, 1994.